

Universitat de Girona

Intelligent Field and Robotic Systems Master

Lab2- Feature Extraction and Image Registration

Prepared by:

Davina Sanghera u1100402

Rihab Laroussi u1100330

Date: November 11, 2024

Contents

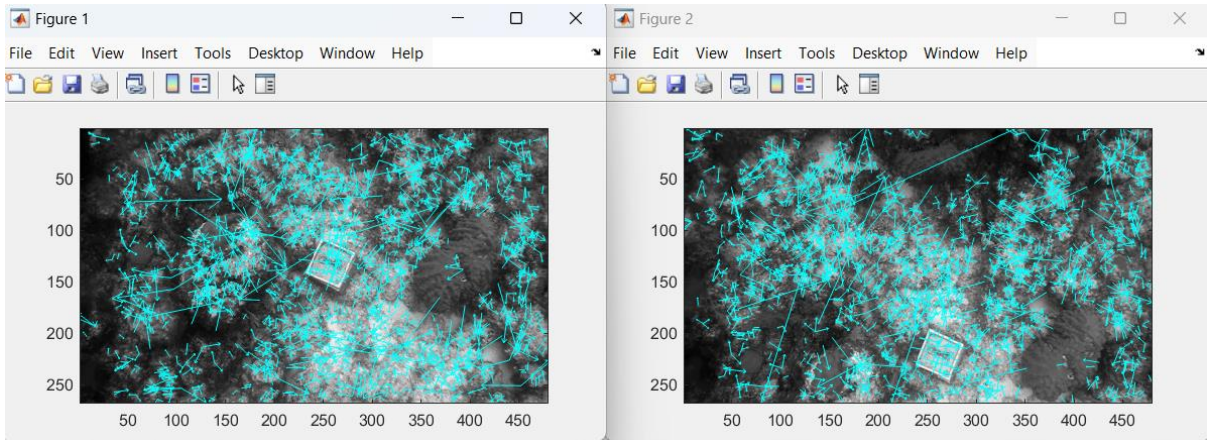
1. Objective	3
2. Test data and auxiliary code	4
3. Feature Association.....	4
4. Estimating a Homography to Register a Pair of Images	9
4.1 Visualization and Analysis.....	11
4.1.1 Visualization	12
4.1.2 Analysis.....	13
5. Improving the Registration Accuracy.....	17
6. Conclusion	21

1. Objective

In this lab, our main objective is to get hands-on experience with SIFT (Scale-Invariant Feature Transform) and planar transformations for rigid image registration. We'll learn how to extract invariant features from images and use their descriptors for feature matching. By applying these techniques, we'll compute homographies to align images. Specifically, we'll test SIFT feature extraction on underwater images, register pairs of images showing the same area from different locations, implement various image motion models using homography matrices, and work on improving registration accuracy through outlier rejection during homography estimation. This will help us understand the practical strengths and weaknesses of using local feature-based approaches in image processing.

2. Test data and auxiliary code

We have used the script `lab2_section2_example.m` to visualize the pair of images and corresponding SIFT features.



Q2.1 What is the meaning of the arrow's directions and lengths?

The two figures display the distinctive keypoints that SIFT detects in each image, represented by arrows. The direction of each arrow indicates the dominant orientation of the keypoint, determined by the local image gradients. The length of the arrow corresponds to the scale at which the keypoint was detected.

3. Feature Association

In this section we will see how the criteria for associating the descriptors can influence the quality of the matched descriptors.

Q3.1 Check the function `match.m` provided by Lowe. What is the purpose of distance ratio (`distRatio`) and how is it used for the feature association?

The distance ratio (`DistRatio`) is used as a threshold to ensure that only high-confidence matches are kept. This is basically a filtering step. It works by comparing the distance (or angle, in our case) between a keypoint's descriptor in the first image and its closest and second closest matches in the second image. If the closest match is significantly better than the second closest then the match is considered reliable; otherwise, it is discarded.

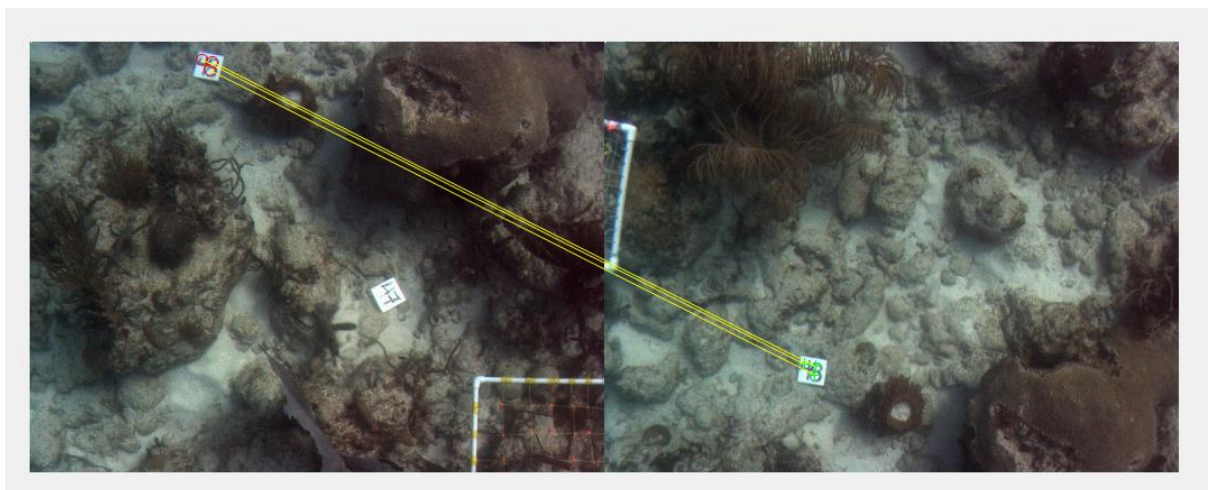
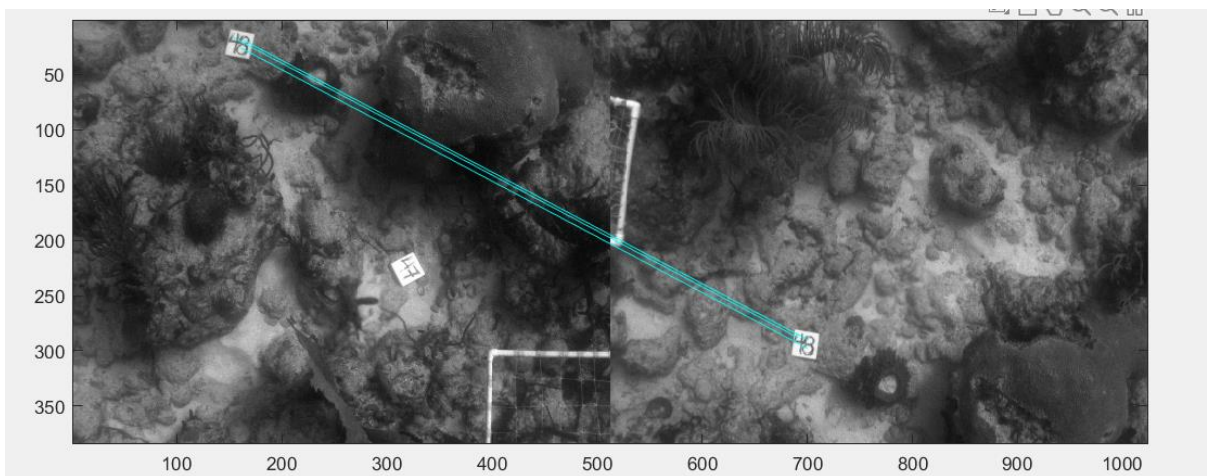
In our case, distRatio is 0.6, which means that only matches where the closest match is at least 40% better than the second closest match will be accepted.

Step 1:

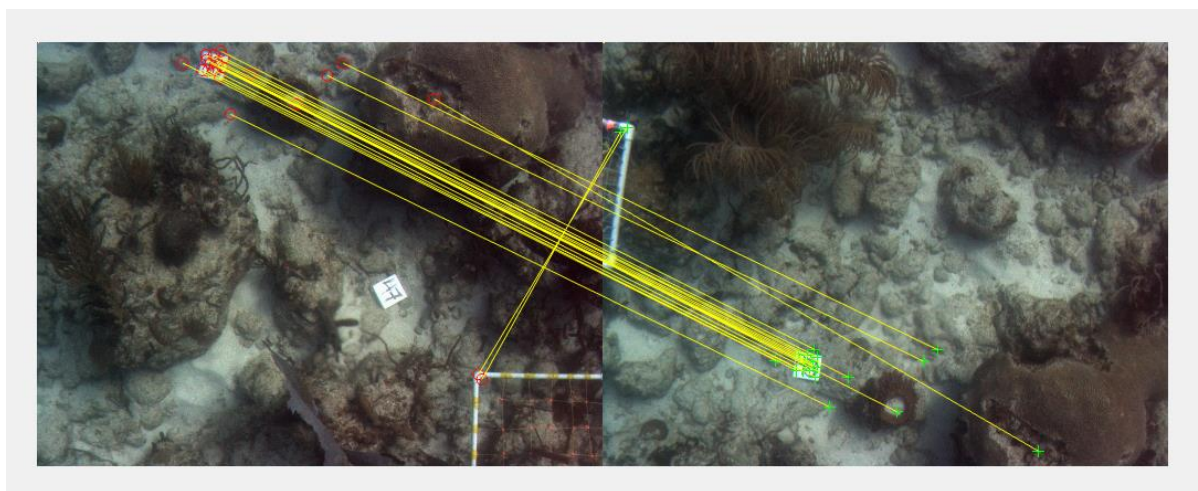
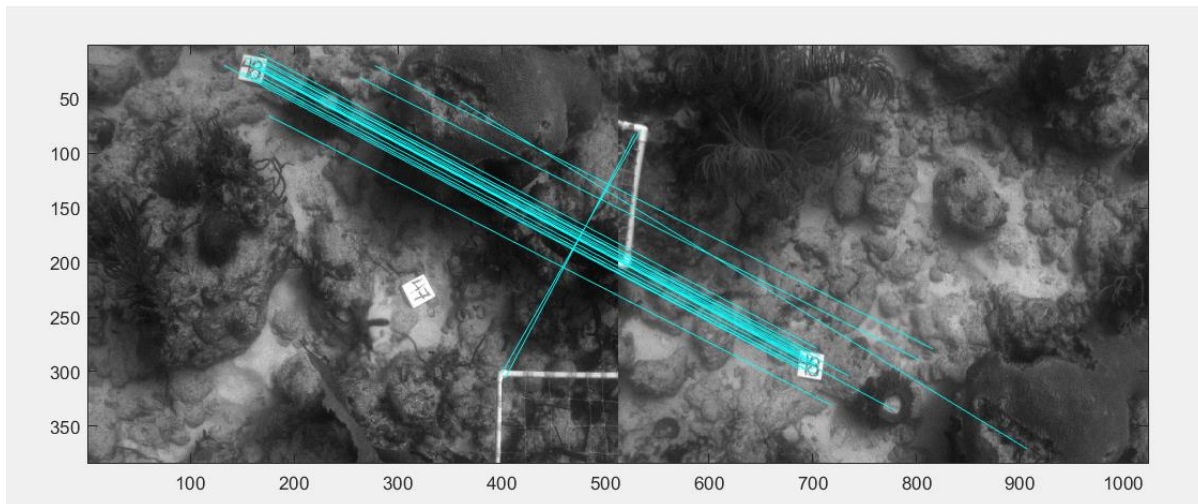
For values of distRatio of 0.4, 0.6 and 0.8, we test the function match siftmodif.m for images imgl01311.jpg (as image1) and imgl01396.jpg (as image 2). We have used the code provided in lab2_section2_example.m to plot the results.

The results are shown as follows:

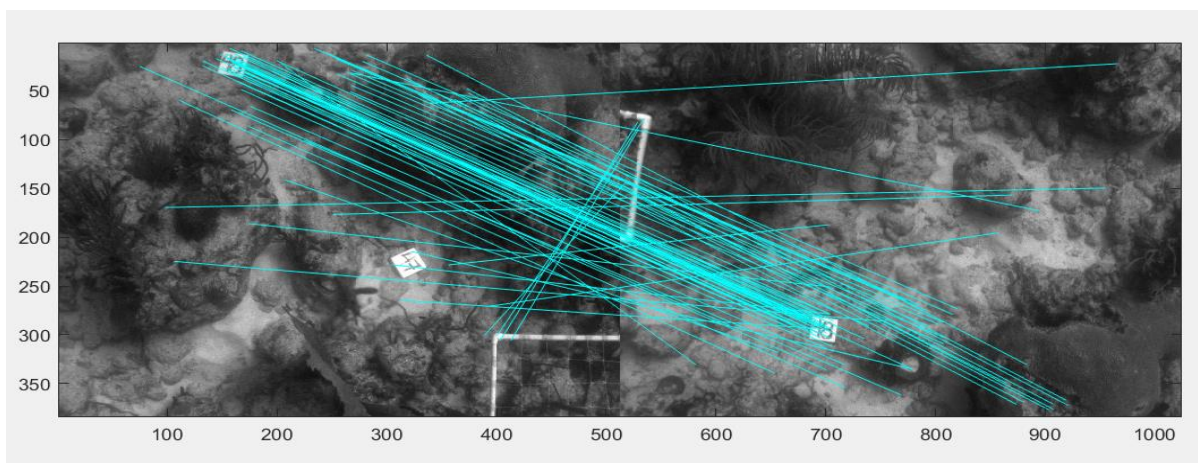
For distRatio = 0.4

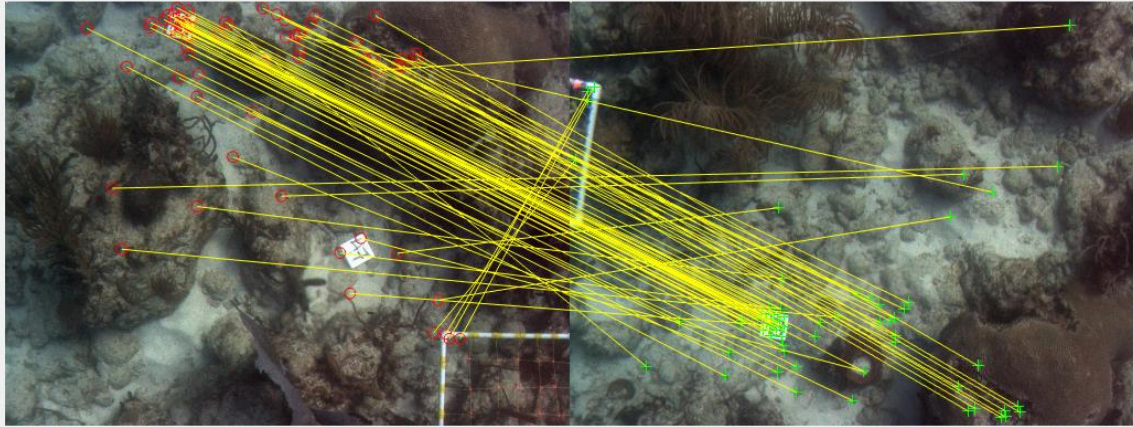


For distRatio = 0.6



For distRatio = 0.8





Q3.2 What is the effect of the parameter `distRatio` in terms of number of associations? How can you tell visually that an association is wrong or not? What is the effect of this parameter in terms of wrong associations?

As the `distRatio` value increases from 0.4 to 0.8, the number of matches generally increases. This is expected because a higher `distRatio` allows more matches by being less strict in distinguishing between the closest and second-closest feature matches.

Visually, incorrect associations often appear as lines that do not align consistently with the general direction of other matches. As we can see in the test with `disRatio` = 0.8, the lines connecting keypoints became less consistent, notice some more mismatched lines, indicating an increase in false positives. The higher threshold allowed closer descriptors that may not represent the same feature to be matched.

Step 2:

In this step, we evaluate how the distance ratio (`distRatio`) affects feature matching accuracy between two images with a known homography. By varying `distRatio` from 0.3 to 0.9, we observe its impact on match quality and quantity. Using a projection error vector, we measure the Euclidean distance between corresponding points in each image, analyzing average error, maximum error, and the number of matched features, including those with high errors.

The following is the code used for implementing the `errorVec` function.

```

% This function calculates the Euclidean distance (error) between each
% point in CL1uv and its corresponding projected point in CL2uv using a given homography matrix H12.

function errorVec = projectionerrorvec(H12, CL1uv, CL2uv)

n = size(CL1uv, 1);           % Number of points in CL1uv (and CL2uv)

errorVec = zeros(n,1);        % Initialize error vector to store distances

for i = 1:n

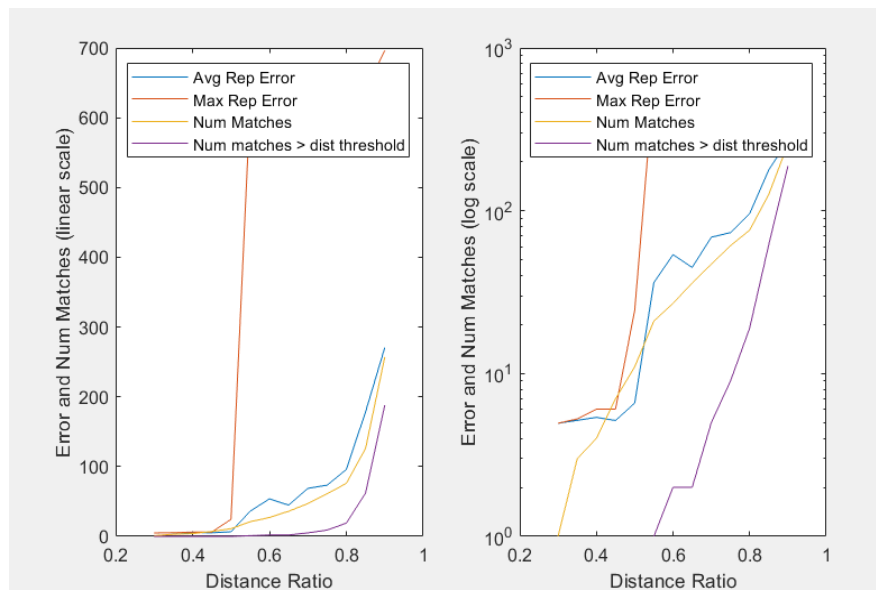
    projection_p = H12 * [CL2uv(i,:), 1]'; % Convert to homogeneous coordinates and apply H12
    projection_p = projection_p(1:2) / projection_p(3); % Normalize the projected point to non-homogeneous coordinates
    errorVec(i) = norm(CL1uv(i,:) - projection_p'); % Compute Euclidean distance (error) between the original point in CL1uv
                                                    % and the projected point in CL2uv

end

end

```

Using the provided script `lab2_section3_empty.m`, we get the following plots:



Q3.3

Interpreting the Results:

From the plot, we observe that the average and maximum projection errors change with `distRatio`. A lower `distRatio` typically yields fewer but more accurate matches, resulting in lower projection errors.

Regarding the number of associated features, increasing `distRatio` generally increases the number of matches. However, this increase also introduces more incorrect

matches, leading to higher projection errors. As a result, a higher number of features with errors above the threshold (50 pixels) indicates more outliers in the matches. This suggests that increasing distRatio beyond a certain point can reduce the reliability of the associations.

- Is it possible to select a suitable value for the distance ratio for these images?

Yes, it is possible. From the plot, we consider a moderate value around 0.4 to 0.6 to be suitable, as it provides a fair number of matches without excessively increasing the average and maximum errors.

- Is it possible to select a value for the distance ratio that guarantees no outliers, while providing a high percentage of associations?

No, it is not possible. Selecting a single distRatio value that completely eliminates outliers while providing a high number of matches is unlikely, because lowering its value will also significantly reduce the number of matches; conversely, increasing distRatio allows more associations but also introduces more outliers. Instead, using additional outlier rejection techniques like RANSAC on top of distRatio can help improve reliability without sacrificing match count.

4. Estimating a Homography to Register a Pair of Images

For each pair of images, we computed all four types of homography transformations: translation, similarity, affine, and projective. The code for this is given below.

```

function H12 = computeHomography(CL1uv, CL2uv, Model)
% COMPUTEHOMOGRAPHY Computes the homography matrix between two sets of points
% based on the specified transformation model.
%
% INPUTS:
% CL1uv - Nx2 matrix of coordinates [u1, v1] in the first image.
% CL2uv - Nx2 matrix of corresponding coordinates [u2, v2] in the second image.
% Model - String specifying the transformation model: 'Translation', 'Similarity', 'Affine', or 'Projective'.
%
% OUTPUT:
% H12 - Homography matrix describing the planar transformation.

switch Model
case 'Translation'
    % Translation Model
    % Computes the translation in the x and y directions.
    tx = mean(CL1uv(:,1) - CL2uv(:,1));
    ty = mean(CL1uv(:,2) - CL2uv(:,2));
    % Construct the translation homography matrix
    H12 = [1 0 tx; 0 1 ty; 0 0 1];

case 'Similarity'
    % Similarity Model
    % Assumes uniform scaling (s), rotation (θ), and translation (tx, ty)
    % The transformation equations are:
    % u2 = s * (cos(θ) * u1 - sin(θ) * v1) + tx
    % v2 = s * (sin(θ) * u1 + cos(θ) * v1) + ty
    N = size(CL1uv, 1);
    A = zeros(2*N, 4);
    B = zeros(2*N, 1);

    for i = 1:N
        u1 = CL1uv(i,1); v1 = CL1uv(i,2);
        u2 = CL2uv(i,1); v2 = CL2uv(i,2);
        A(2*i-1,:) = [u2 -v2 1 0];
        A(2*i,:) = [v2 u2 0 1];
        B(2*i-1) = u1;
        B(2*i) = v1;
    end

    % Solve the system of equation
    Q = A \ B;
    a = Q(1); b = Q(2); tx = Q(3); ty = Q(4);
    % a = s * cos(θ)
    % b = s * sin(θ)
    H12 = [a -b tx; b a ty; 0 0 1];

case 'Affine'
    % Affine Model
    % Accounts for linear transformations including scaling, rotation, translation, and shear.
    % The transformation equations are:
    % u2 = a*u1 + b*v1 + tx
    % v2 = c*u1 + d*v1 + ty
    N = size(CL1uv, 1);
    A = zeros(2*N, 6);
    B = zeros(2*N, 1);

    for i = 1:N
        u1 = CL1uv(i,1); v1 = CL1uv(i,2);
        u2 = CL2uv(i,1); v2 = CL2uv(i,2);
        A(2*i-1,:) = [u2 v2 1 0 0 0];
        A(2*i,:) = [0 0 0 u2 v2 1];
        B(2*i-1) = u1;
        B(2*i) = v1;
    end

    % Solve the system of equations
    Q = A \ B;
    a = Q(1); b = Q(2); tx = Q(3);
    c = Q(4); d = Q(5); ty = Q(6);
    % Construct the affine homography matrix
    H12 = [a b tx; c d ty; 0 0 1];

```

```

case 'Projective'
    % Projective Model
    % The transformation equations are:
    % u2 = (a*u1 + b*v1 + tx) / (g*u1 + h*v1 + 1)
    % v2 = (c*u1 + d*v1 + ty) / (g*u1 + h*v1 + 1)
    N = size(CL1uv, 1);
    A = zeros(2*N, 8);
    B = zeros(2*N, 1);

    for i = 1:N
        u1 = CL1uv(i,1); v1 = CL1uv(i,2);
        u2 = CL2uv(i,1); v2 = CL2uv(i,2);
        A(2*i-1,:) = [u2 v2 1 0 0 0 -u2*u1 -u2*v1];
        A(2*i,:) = [0 0 0 u2 v2 1 -v2*u1 -v2*v1];
        B(2*i-1) = u1;
        B(2*i) = v1;
    end

    % Solve the system of equations
    Q = A \ B;
    a = Q(1); b = Q(2); tx = Q(3);
    c = Q(4); d = Q(5); ty = Q(6);
    g = Q(7); h = Q(8);
    % Construct the projective homography matrix
    H12 = [a b tx; c d ty; g h 1];

otherwise
    % Handle invalid models
    warning('Invalid model, returning identity homography');
    H12 = eye(3);
end
end

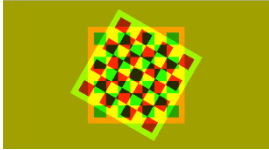
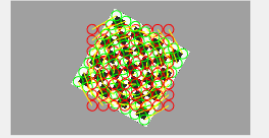
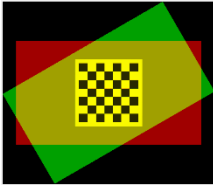
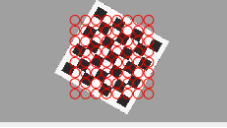
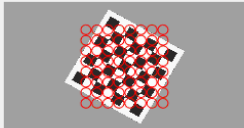
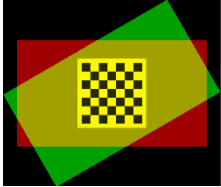
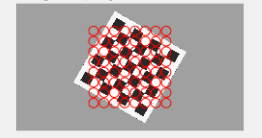
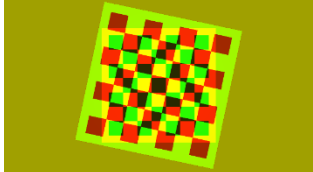
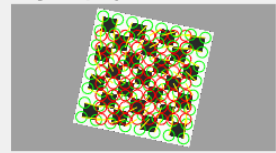
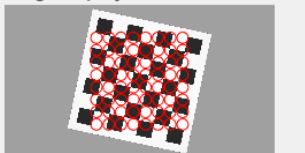
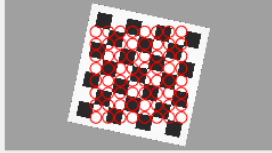
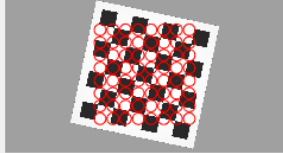
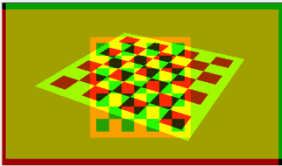
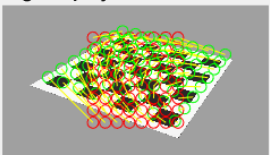
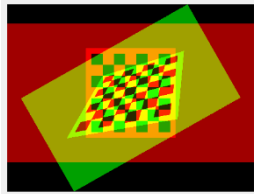
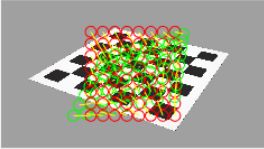
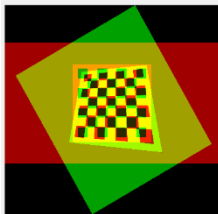
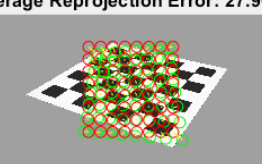
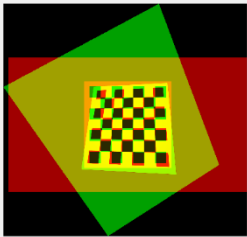
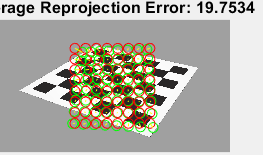
```

The goal was to identify the motion model that best fits the transformation for each pair of images using the `showwarpedimages(I1, I2, H12)` function for visual verification.

4.1 Visualization and Analysis

In this section, we will present the output results of the homography for each pair of images and determine which homography transformation provides the best alignment. To achieve this, we used a function called `visualizeReprojectionError` to visually assess the reprojection errors associated with each transformation. This function plots the original and reprojected points, along with the corresponding errors, allowing us to evaluate the accuracy and effectiveness of the homography matrices applied to each image pair. By comparing the reprojection errors, we can identify the transformation that offers the most precise alignment.

4.1.1 Visualization

Image Pair	Translation	Similarity	Affine	Projective
00.png and 01.png	 Reprojected Image with Errors Average Reprojection Error: 138.0211 	 Reprojected Image with Errors Average Reprojection Error: 0.2319 	 Reprojected Image with Errors Average Reprojection Error: 0.2321 	 Reprojected Image with Errors Average Reprojection Error: 0.2301 
00.png and 02.png	 Reprojected Image with Errors Average Reprojection Error: 88.0939 	 Reprojected Image with Errors Average Reprojection Error: 0.1595 	 Reprojected Image with Errors Average Reprojection Error: 0.1600 	 Reprojected Image with Errors Average Reprojection Error: 0.1589 
00.png and 03.png	 Reprojected Image with Errors Average Reprojection Error: 204.7935 	 Reprojected Image with Errors Average Reprojection Error: 86.0312 	 Reprojected Image with Errors Average Reprojection Error: 27.9692 	 Reprojected Image with Errors Average Reprojection Error: 19.7534 

4.1.2 Analysis

For Image Pair [00.png](#) and [01.png](#):

- *Most Accurate Transformation:*

Similar to the similarity transformation and the affine transformation, the projective transformation has the lowest average reprojection error (0.2301), making it the most accurate method. However, since the similarity transformation is less computationally expensive this would be the preferred transformation in cases of limited computational power.

- *Least Accurate Transformation:*

The translation transformation has the highest average reprojection error (138.0211), indicating it's the least accurate for this task.

For Image Pair [00.png](#) and [02.png](#):

- *Most Accurate Transformation:*

Again, very close to the similarity transformation and the affine transformation, the projective transformation has the lowest average reprojection error (0.1589), suggesting it's the most accurate in aligning the points between the two images. But again, if efficiency is an important factor, then the similarity transformation would be the better option.

- *Least Accurate Transformation:*

The translation transformation has the highest average reprojection error (88.0939), it is the least effective due to its simplicity and inability to handle complex distortions.

For Image Pair [00.png](#) and [03.png](#):

- *Most Accurate Transformation:*

The best alignment with the lowest reprojection error (19.7534), demonstrating that handling perspective distortion is crucial for accurate transformation.

- *Least Accurate Transformation:*

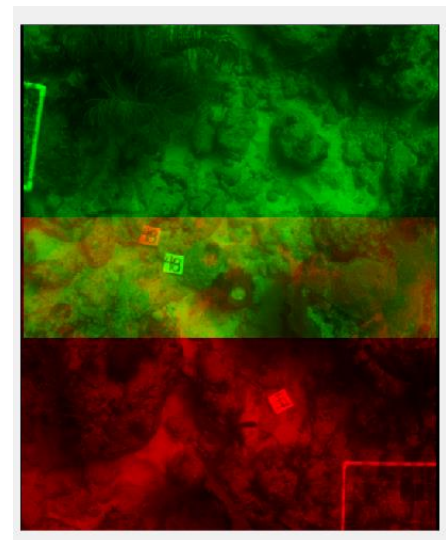
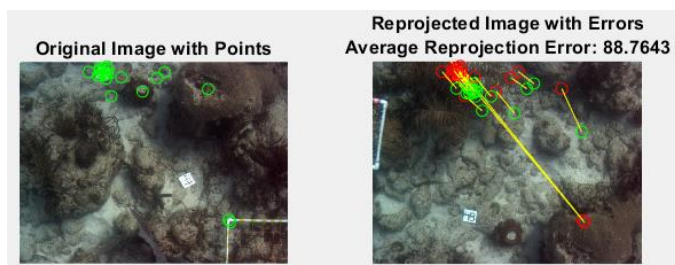
The translation transformation has the highest average reprojection error (204.7935), showing that simple translation is not sufficient for complex alignments.

Step 3:

In this section, we generated lists of matched points for 'imgl01311.jpg' and 'imgl01396.jpg,' using the ``matchsiftmodif.m`` function. By setting the ``distRatio`` parameter to 0.6, we ensured more reliable point matching. We then computed the corresponding homography matrices for these image pairs and displayed the results.

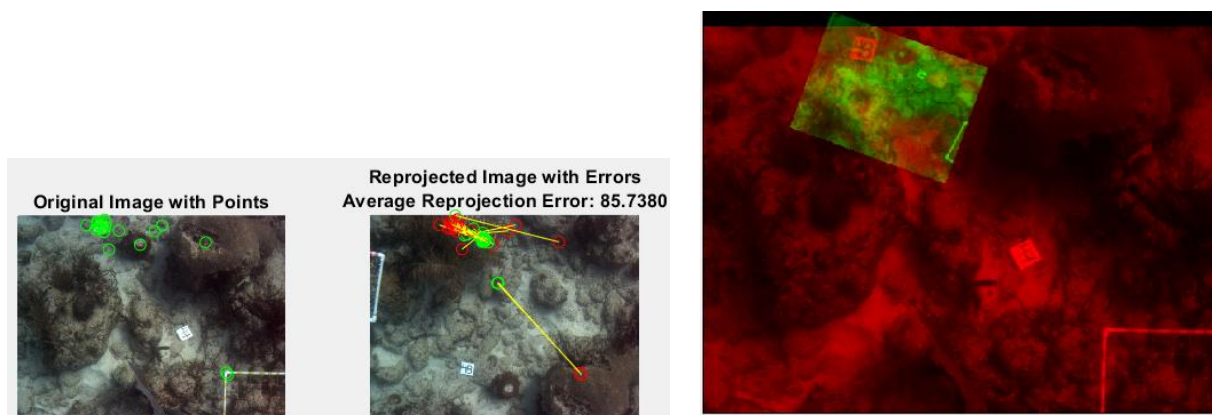
For the Translation Model:

The transformation method applied is not sophisticated enough to handle the distortions present in the images. The high average reprojection error suggests significant misalignment between the original and reprojected points.



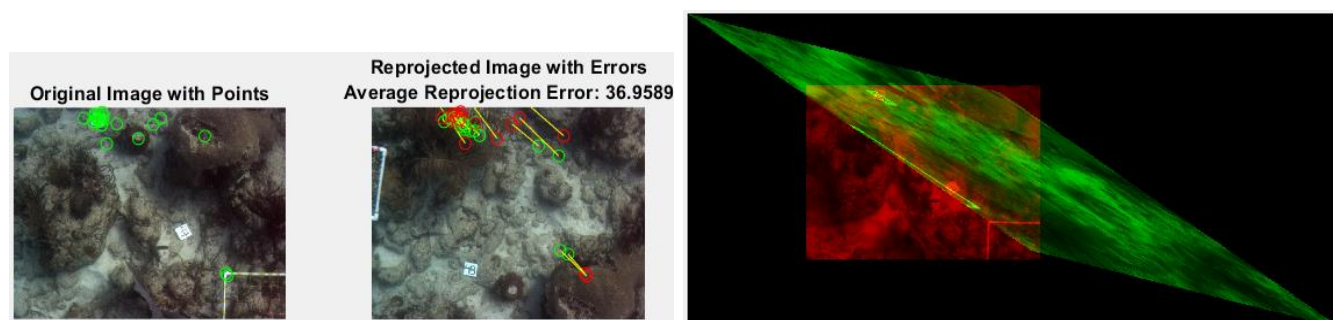
For the Similarity Model:

The similarity transformation has effectively improved the alignment of points, reducing the average reprojection error to 85.7380. This indicates better performance compared to a simple translation but still leaves room for improvement.



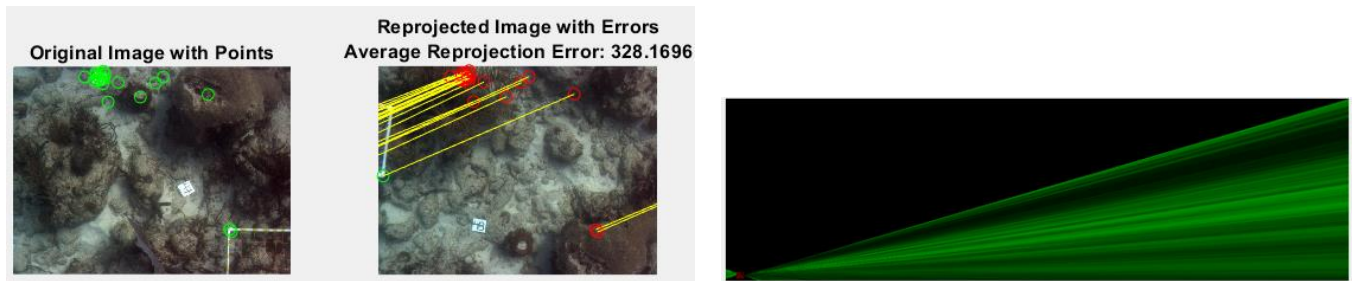
For the Affine Model:

The affine transformation has effectively reduced the reprojection errors, achieving an average reprojection error of 36.9589. This demonstrates its ability to handle more complex geometric distortions compared to simpler transformations like translation and similarity.



For the Projective Model:

The output indicates significant errors and misalignment in the projective transformation. Despite its flexibility, the projective transformation didn't produce a lower reprojection error in this case.



Q4.1 Did you notice any benefit or drawback using motion models with more degrees of freedom than is theoretically needed in some image pairs?

When using a motion model with more degrees of freedom than is required, there is a risk of overfitting. This can be seen in the models computed for images 'imgl01311.jpg' and 'imgl01396.jpg', where the reprojection error of the projective transform increases significantly compared to the previous models with less degrees of freedom. However, a higher number of degrees of freedom can be beneficial in some complex examples. E.g. in the models computed for images '00.png' and '01.png', the transformation is more complex and so the projective model is better since its higher degrees of freedom gives it the flexibility needed to capture these complexities.

Q4.2 Does the obtained homography matrices allow registering the image pairs accurately, or are there significant misalignments?

Although the affine homography produces a lower average reprojection error relative to the other homography models, a reprojection error of 36.9589 is still quite high. Whilst the simpler image transformations were easier to compute the homography for, the more complex example depicted above requires us to implement further functions in order to obtain an accurate homography matrix. This is because the images above are real world images that contain a lot of noise. Therefore, when the correspondences are computed for these images, many outliers are returned. These outliers are then included in the correspondence sets when computing the homography, thus leading to high reprojection errors when these homographies are used to project the points from one

image to another. Therefore, in order to improve the registration accuracy, we should implement an algorithm that is able to remove these outliers as a preprocessing step.

5. Improving the Registration Accuracy

Step 1:

While the implemented homographies produce relatively low average reprojection errors for the less complex image transformations, the effect of outliers in the data means that the homography using these false data associations and the homography therefore struggles to define a highly accurate transformation between the images. Therefore, the RANSAC algorithm is implemented so that these outliers can be removed prior to the computation of the homography matrix.

The implemented `computeHomographyRANSAC.m` function is given below:

```

function [H12, INLIERS1uv, INLIERS2uv] = computeHomographyRANSAC(CL1uv, CL2uv, Model)
% computeHomographyRANSAC : perform RANSAC to remove outliers then estimate the Homography between two images

% Initialise variables
dist_threshold = 50; % distance threshold
sample_size = 10; % sample size
no_of_trials = 100; % number of trials
term_threshold = (1 - 0.2) * no_of_trials; % termination threshold
no_of_points = height(CL1uv); % number of points

% Ensure sample size is not greater than the number of points
if sample_size > no_of_points
    sample_size = no_of_points;
end

best_consensus_idx = [];
best_consensus_size = 0;
best_H12 = eye(3);

for i = 1:no_of_trials
    % Randomly sample points
    sample_idx = randperm(no_of_points, sample_size);
    sample1 = CL1uv(sample_idx, :);
    sample2 = CL2uv(sample_idx, :);

    % Estimate H12
    H12 = computeHomography(sample1, sample2, Model);

    % Calculate projection error of each correspondence
    errorVec = projectionerrorvec(H12, CL1uv, CL2uv);

    % Inliers are those with an error below the distance threshold
    inliers_idx = find(errorVec < dist_threshold);

    % Check if this is the largest consensus
    % and if the current model is above the termination threshold
    consensus_size = length(inliers_idx);
    if consensus_size > best_consensus_size
        best_consensus_idx = inliers_idx;
        best_consensus_size = consensus_size;
        best_H12 = H12;
    end
    if (consensus_size / no_of_points) >= term_threshold
        break
    end
end

% Separate inliers and outliers based on the best consensus set found
H12 = best_H12;
INLIERS1uv = CL2uv(best_consensus_idx, :);
INLIERS2uv = CL1uv(best_consensus_idx, :);
end

```

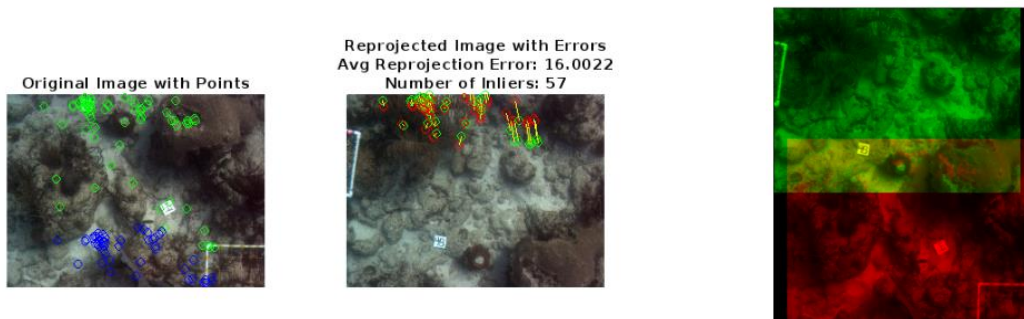
To begin with, the parameters of the algorithm are set. The distance threshold here has the same trade off as the distance ration. If the distance threshold is higher, then the RANSAC algorithm will allow correspondences with higher reprojection errors to be defined as inliers, therefore adding more false correspondences to the computation of the homography. On the other hand, if the distance threshold is lower, then the number of correspondences that are accepted as inliers is lower and thus, we use fewer correspondences to compute the homography. Based on the image pair: 'imgl01311.jpg' and 'imgl01396.jpg', a distance threshold of 150 was set so that a sufficient number of correspondences remained whilst the reprojection error decreased significantly. The remaining parameters were given standard values.

In the main for loop, a subset of 10 correspondences is selected from CL1uv and CL2uv. The desired homography is then computed using the computeHomography function and the projection error of this homography is computed using the projectionerrorvec function. Then points with a projection error less than the distance threshold are added to the list of inliers. If the number of inliers found in the current iterations is the highest number of inliers found so far, then this consensus becomes the best consensus. After 100 trials, the best consensus is returned along with the homography matrix that was computed for this consensus.

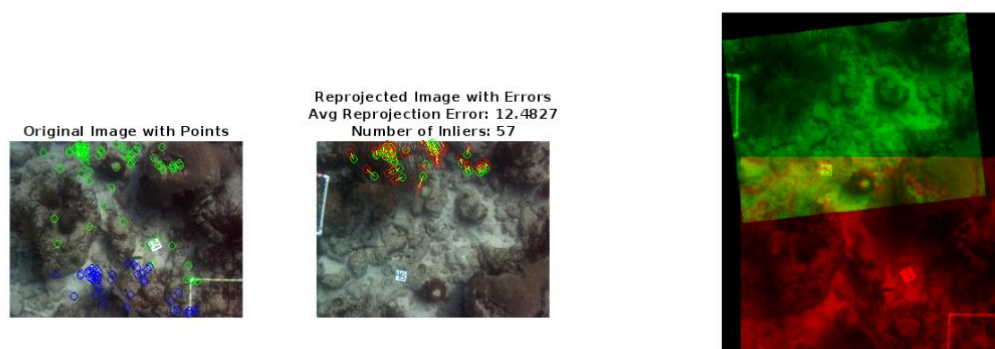
Step 2:

With the RANSAC algorithm added to the pipeline implemented, we can now compare the average reprojection errors for the 'imgl01311.jpg' and 'imgl01396.jpg' image pair, that we got before using RANSAC and after using RANSAC.

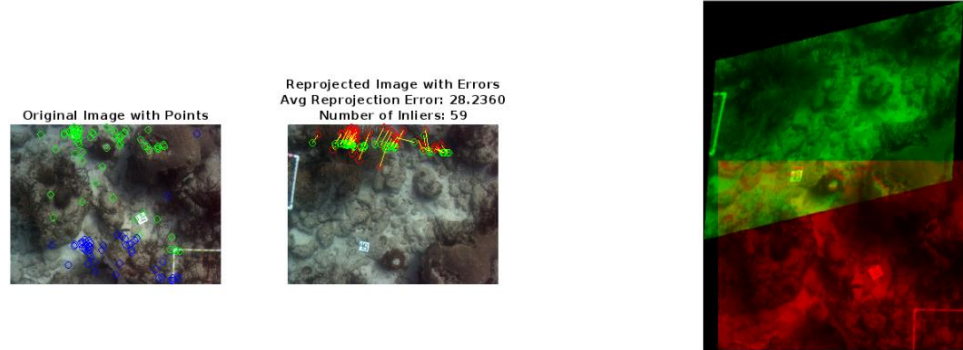
For the Translation Model:



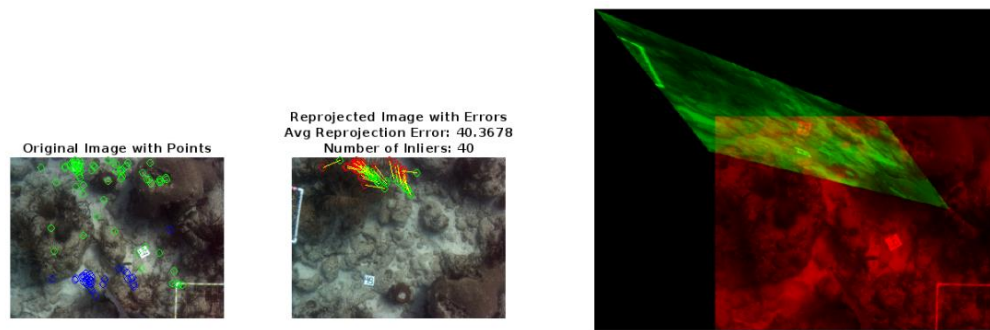
For the Similarity Model



For the Affine Model:



For the Projective Model:



Q5.1 Are the results obtained with the RANSAC-based homography estimation more accurate than the previously obtained?

In all cases, the reprojection error decreases when the RANSAC algorithm is applied. In particular, the transformation and projective transformations show significant improvement when outliers are rejected before computing the rejection matrix. This makes sense since the projective transformation can be very sensitive to outliers, with the similarity and affine transformations being less sensitive. As is detailed above, the number of correspondences (inliers) is reduced however a significant number remain. Again, if the distance threshold is increased, more correspondences will be accepted at the cost of higher average reprojection errors. Whereas if the distance threshold is decreased, even better average reprojection errors should be achieved. However, this

parameter is dependent on the images because different images have different levels of noises and different numbers of outliers.

6. Conclusion

This lab demonstrated the effectiveness of SIFT-based feature extraction and homography transformations for image registration, particularly in underwater images. By tuning the distance ratio, we balanced match accuracy with outlier rejection, finding that a moderate ratio (0.4-0.6) provided reliable matches. Different homography models were tested, with projective transformations offering precise alignments for complex scenes, while simpler models like similarity were more efficient in straightforward cases. Adding RANSAC improved accuracy by rejecting outliers, reducing reprojection errors and enhancing homography reliability. Overall, this lab highlighted the strengths and limitations of local feature-based methods for accurate image alignment.